High-Level Design: Agentforce for Setup v2

Status: Socialization
Authors: Stephan Hesmer Anirudh Srinivas Alan Khumsaard Padmaraj Moorthy Shubham Yelekar Ivan Minevskiy
Feature: Agentforce for Setup v2
Last Updated: Jan 16, 2026

Abstract

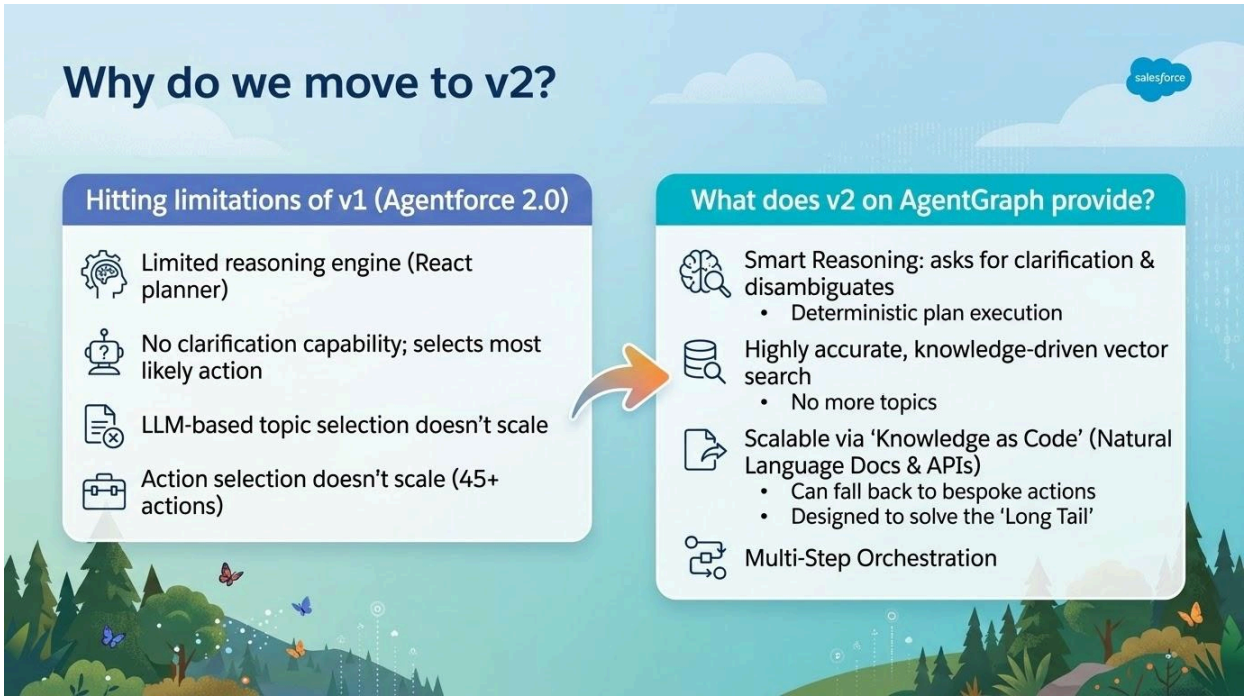
Agentforce for Setup v2 represents a strategic evolution from the reactive, topic-based v1 architecture to a proactive, knowledge-driven "Smart Consultant." One of the main goals is to solve the "Long Tail" of administrative tasks-moving beyond high-frequency "hero" use cases to cover a vast spectrum of niche administrative needs. Another goal is to create an open-ended agent that will eventually be able to provide meaningful guidance to the administrator based on org information, providing a personalized “architect-in-a-box”. This is achieved by implementing a more advanced reasoning agent along the lines of “Deep Research” by leveraging a stateful Agent Graph (Daisy + AgentGraph DSL). This architecture is utilizing a multi-layered Knowledge Base of Metadata & API Grounding, Product Documentation, Natural Language Documents (NLDs) and a set of consolidated Generic Actions to provide deterministic, trustworthy, and scalable setup automation.

Overview

Agentic Setup’s charter focuses on **reducing customer Time to Value and cost of maintenance** by expediting:

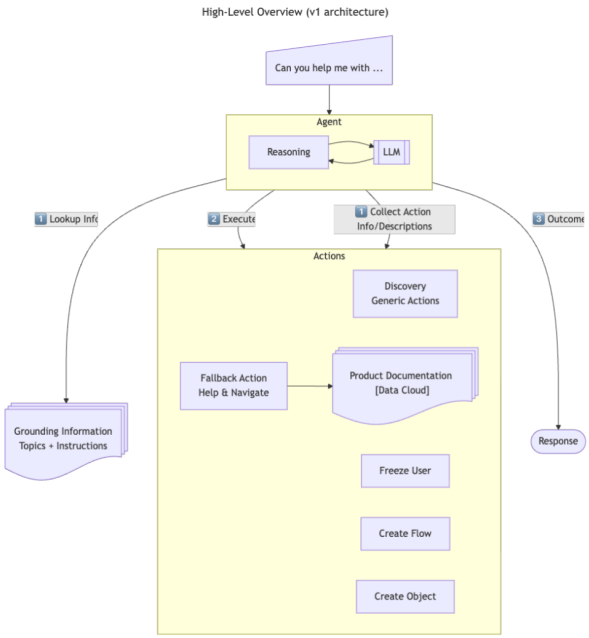
- Configuration creation & management (think: user access, data models, identity)
    - Data management
    - Troubleshooting
- Product/feature enablement (think: Service Cloud Email-to-Case, ITSM Setup)
- Guidance & optimizations (think: LEX performance, Storage & Licensing, Security Scores)

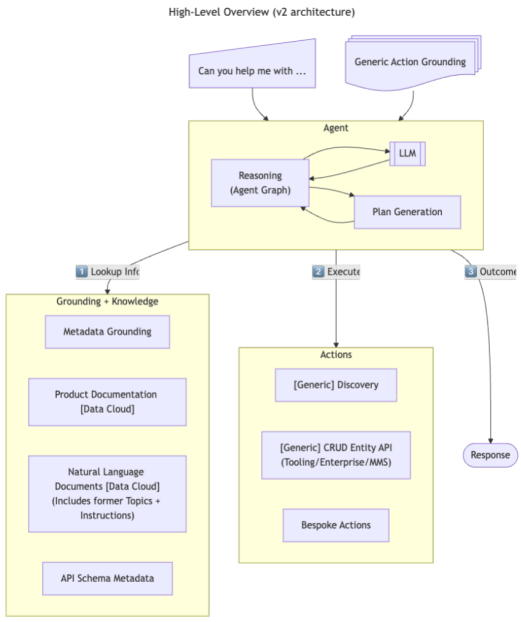
What is v2 and why do we move to a new architecture?



The move to the v2 architecture is necessitated because of the limitations of the topics & instructions model in Agentforce 2.0 paired with bespoke java actions. The pilot already has 45 bespoke actions with a small number of generic actions and as we envision to scale out to many more scenarios and entities across all of Setup, we need a drastically different approach to be able to execute on this vision.

This diagram provides a side-by-side overview of v1 vs v2 to show the main differences.





Agentforce for Setup v2 leverages Agentforce 3.0 (Daisy + Agent Graph) for the runtime execution and Data Cloud for the storage and retrieval of the Knowledge Base. It interacts with the Platform Metadata layer via the Grounding Service to understand org configuration and executes changes via a combination of generic actions and a few select bespoke actions.

Business Drivers

- The "Long Tail" Problem: v1 focused on top-tier "hero" use cases. v2 aims to solve the 30-40% of use cases that represent over 90% of user needs but are too numerous to hard-code.
- Reliability: Leverage Agent Graph's capabilities to provide a more reliable and flexible agent.
- Scalability: Enable "Knowledge as Code." Adding new capabilities should require adding a Markdown file (NLD), not writing new Java code.

Design Goals

- Deterministic Execution: Create a hard boundary between the "Thinking" (Non-Deterministic) and "Doing" (Deterministic) phases. Always keep the human in the loop before any modification action is performed.
- Hybrid Intelligence: Support a "Fast Path" for known tasks and a "Slow Path" for novel problems.
- Trust: 100% fidelity in plan execution; if the agent builds a plan, it must execute exactly as described.

Out of Scope

- Existing Bespoke Actions: Highly complex, bespoke UI rendering or mission-critical logic that requires custom Java actions will remain and not be replaced by Generic Actions at the moment.
- Full "Ask Me Anything" Agent: While v2 supports “architect-in-the-box” informational queries, the primary focus is on the ability to accelerate more capabilities. More advanced agent capabilities will be addressed later.

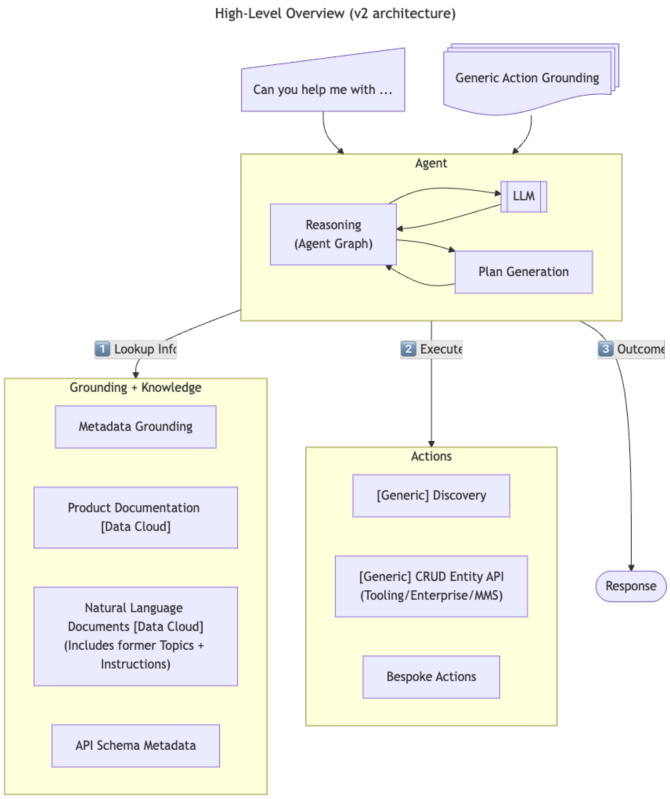
Demos

- Executive Demo as of Dec 4th (3:33min)
    - Slides to explain pilot, v2, Natural Language Documents, plus a demo
- Longer Demo as of Nov 14th (7:58min)
    - Freezing a user using generic actions
    - Bespoke Clone User action
    - Information-seeking requests
    - Simple org-level user detail lookup

Conceptual Design

Component Model

The main components of the v2 architecture are:



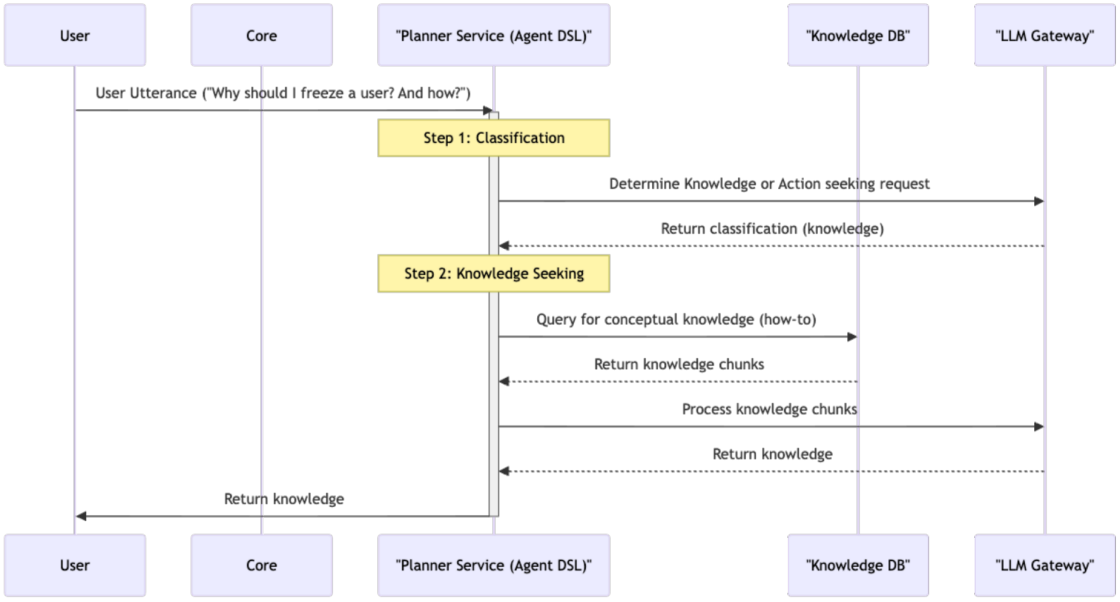
1. **Knowledge Base**: This is the foundation of the entire system. Compared to the existing pilot, the agent is querying for knowledge *first* before trying to engage in any action or plan. It involves creating a robust, multi-layered knowledge base and data stores into a vector database. This provides the LLM with the raw material to understand user intent.
2. **Smart Agent**: The agent takes the contextual information gathered from the knowledge base and translates it into a concrete, executable plan.
3. **Actions**: The system provides a set of generic, CRUD API actions to execute the plan alongside more specialized / bespoke ones as needed based on the complexity of the operation.

Flow & Examples

This section describes two main flows through the system. One knowledge seeking, and one action seeking.

Knowledge Seeking Scenario

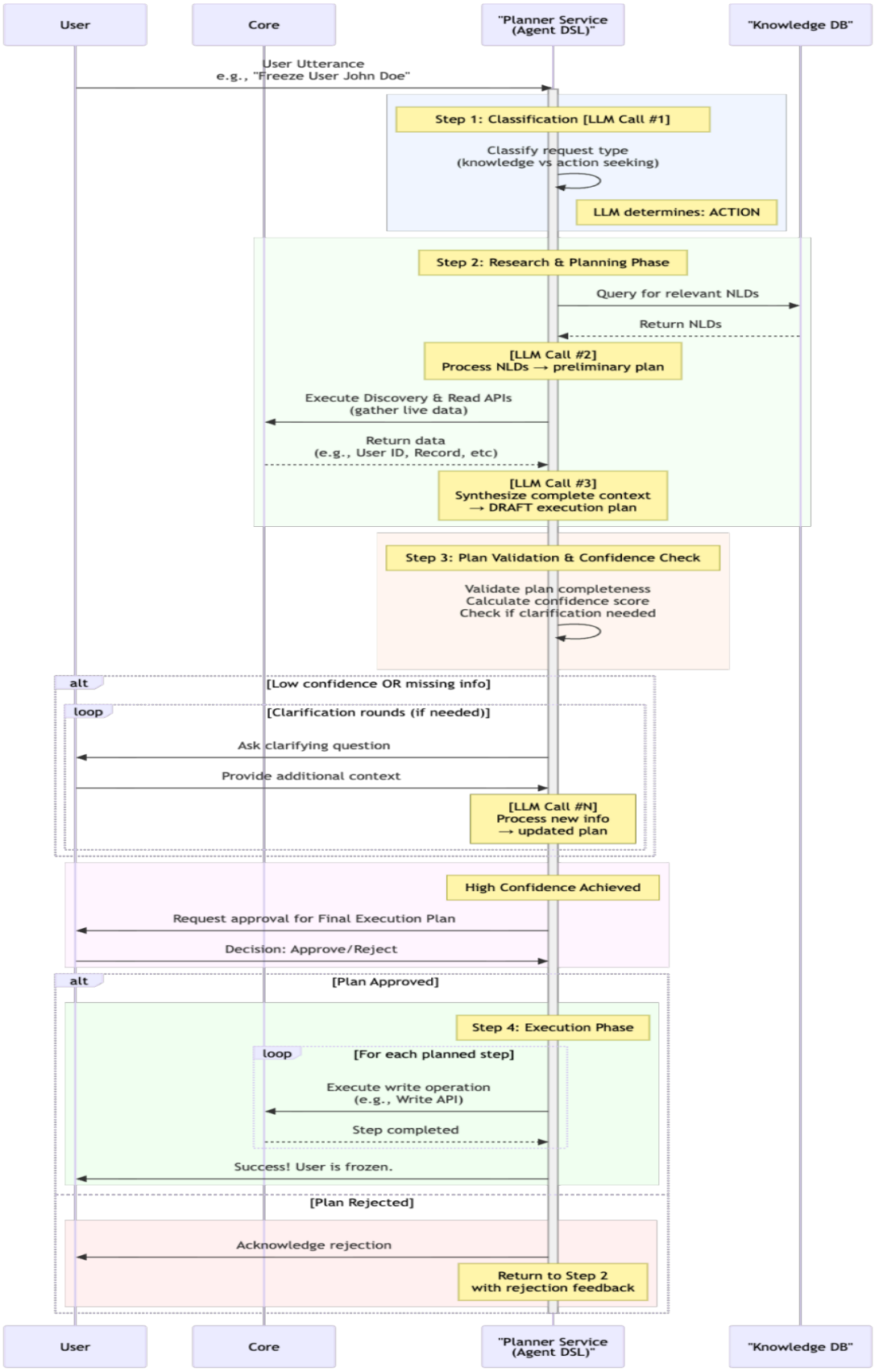
- **User Request:** The user submits an informational query (e.g., "Why should I freeze a user?").
- **Classification:** The agent triages the request as "Knowledge seeking" rather than "Action seeking."
- **Retrieval / Knowledge Node:** The `ConversationManager` supervises the `Knowledge Node`. The `Knowledge Node` is responsible for retrieving information from the appropriate source.
    - **For a Conceptual Query:** It performs a semantic search against its indexed **Product Documentation** and **Best Practices NLDs** to find the most relevant articles and knowledge chunks.
    - **For a Data Query:** It uses its **tenant-aware grounding tools** to query the live Salesforce org for the specific data the user requested.
- **Processing & Response:** These raw knowledge are processed and synthesized into a coherent answer to the user.



A more detailed example can be found here.

Action Seeking Scenario

- **User Request:** The user issues a command (e.g., "Freeze User John Doe").
- **Classification:** The agent triages the request as "Action seeking" rather than "Knowledge seeking."
- **Research & Planning:**
    - The agent retrieves Natural Language Documents from the Knowledge Base to form a preliminary plan.
    - As needed, it fetches live data from the core system with the goal to create a plan to present to the user
- **Validation:** The agent validates the plan and calculates a confidence score. If the plan requires a clarification, the system asks the user and loops back to the research phase.
- **Approval:** High-confidence plans are presented to the user for final approval.
- **Execution:** The agent executes the approved steps (via generic or bespoke action) and confirms success to the user.

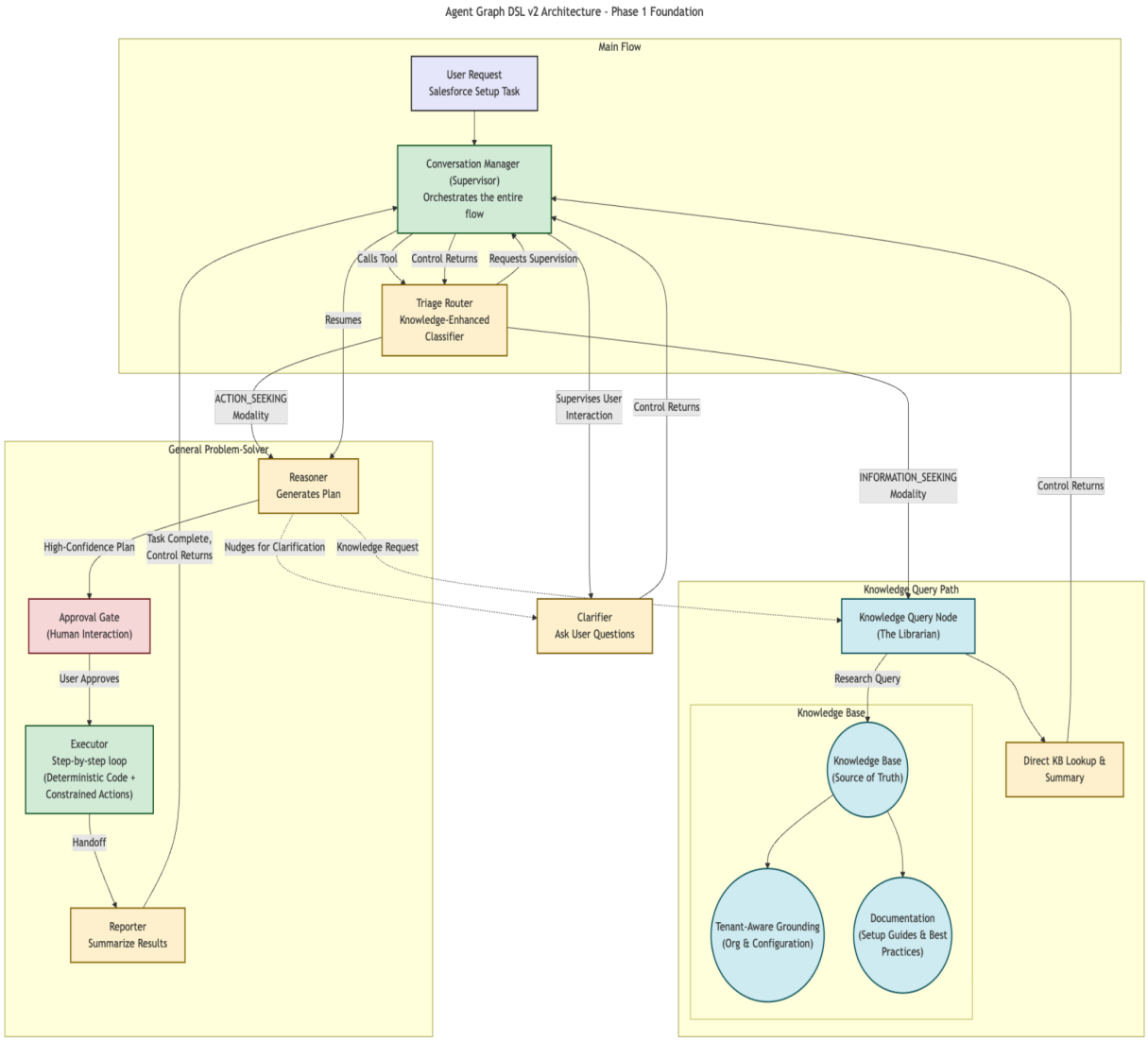

User
Core
"Planner Service (Agent DSL)"
"Knowledge DB"
User Utterance
e.g., "Freeze User John Doe"
Step 1: Classification [LLM Call #1]
Classify request type
(knowledge vs action seeking)
LLM determines: ACTION
Step 2: Research & Planning Phase
Query for relevant NLDs
Return NLDs
[LLM Call #2]
Process NLDs → preliminary plan
Execute Discovery & Read APIs
(gather live data)
Return data
(e.g., User ID, Record, etc)
[LLM Call #3]
Synthesize complete context
→ DRAFT execution plan
Step 3: Plan Validation & Confidence Check
Validate plan completeness
Calculate confidence score
Check if clarification needed
alt
[Low confidence OR missing info]
loop
[Clarification rounds (if needed)]
Ask clarifying question
Provide additional context
[LLM Call #N]
Process new info
→ updated plan
High Confidence Achieved
Request approval for Final Execution Plan
Decision: Approve/Reject
alt
[Plan Approved]
Step 4: Execution Phase
loop
[For each planned step]
Execute write operation
(e.g., Write API)
Step completed
Success! User is frozen.
[Plan Rejected]
Acknowledge rejection
Return to Step 2
with rejection feedback
User
Core
"Planner Service (Agent DSL)"
"Knowledge DB"

A more detailed example can be found here.

Architecture: The Three Main Pillars

The v2 architecture is built upon three distinct pillars that function in sequence to transform a user request into a successful outcome.

Pillar 1: The Agent (Agent Graph & Reasoning)



1.1 A "Supervisor & Specialists" Model

The core of the V2 agent architecture is a migration from a simplistic ReAct loop to a **Stateful Agent Graph**. This new architecture implements a **"Supervisor & Specialists" model** to orchestrate complex tasks, ensure resilience, and provide a trustworthy user experience by separating distinct cognitive loads into specialized components.

The model is composed of a single, persistent **Supervisor** (`ConversationManager`) and a team of single-purpose **Specialists** (`Knowledge Node`, `Reasoner Node`, `Executor Node`).

The Supervisor: `ConversationManager`

- **Role:** The persistent home base and master orchestrator for the entire conversation.
- **Responsibilities:** The `ConversationManager` is the brain of the operation. It manages the master state, handles all user-facing communication, and is responsible for delegating tasks to the appropriate specialist via **Supervision**. Control always returns to the `ConversationManager` after a specialist completes its task, allowing for a resilient, stateful conversational flow.

The Specialists: A Team of Experts

These are the single-purpose nodes that are supervised by the `ConversationManager` to perform specific tasks.

- **`Knowledge Node`:**
  - **Role:** The authoritative source of truth.
  - **Responsibilities:** Responsible for retrieving all types of knowledge, including **Product Documentation**, the various categories of **Natural Language Documents (NLDs)**, and performing **Metadata & API Grounding** queries against the live org.
- **`Reasoner Node`:**
  - **Role:** The central intelligence hub for planning and problem-solving.
  - **Responsibilities:** Responsible for translating a user's goal into a deterministic `executionPlan`. It handles all ambiguity, synthesizes information from multiple NLDs, and runs the **Plan Generation Pipeline**.
- **`Executor Node`:**
  - **Role:** The reliable and predictable execution engine.
  - **Responsibilities:** Responsible for taking a finalized `executionPlan` and executing each of its action calls with 100% fidelity. It operates without reasoning, ensuring that the agent's actions are always predictable.

1.2 The Triage & Delegation Flow: From Intent to Action

The agent's intelligence begins with the ConversationManager's Triage process. This is a fast, efficient mechanism to determine the user's high-level intent and delegate the request to the appropriate specialist.

Step 1: The Triage action (Intent Classification)

The ConversationManager invokes its TriageRequestIntent action. This action's only job is to classify the user's request into one of two fundamental modalities, using concrete examples to make the distinction clear.

- `INFORMATION_SEEKING` **("The user wants to know something.")**
  This modality is for requests where the user's goal is to retrieve and understand information, without modifying the org.
  - **Example 1 (Conceptual Query):** *"How do sharing rules work?"*
  - **Example 2 (Simple Data Query):** *"What is the ID for the 'Sales Team' public group?"*
- `ACTION_SEEKING` **("The user wants the agent to *do* something")**
  This modality is for any request that requires a process to be initiated or the state of the org to be modified. This includes complex, multi-part data queries.
  - **Example 1 (Simple Command):** *"Clone the user John Smith."*

Step 2: The ConversationManager Delegates

Based on the Triage result, the ConversationManager supervises the correct specialist.

- If the modality is `INFORMATION_SEEKING`, the CM supervises the Knowledge Node to find and present a direct answer (either from documentation or a simple, live data query).
- If the modality is `ACTION_SEEKING`, the CM supervises the Reasoner Node to handle the entire planning and execution lifecycle.

Step 3: The Reasoner as a "Synthesis Engine"

When the Reasoner Node is activated for an `ACTION_SEEKING` task, its core function is not simple choice-making, but comprehensive **synthesis**. It analyzes the user's goal and queries the Knowledge Node to gather all relevant NLDs (Workflow, Higher Order, Best Practices). With this knowledge, it strategically decides which of its two primary processing paths to take.

1. The High-Velocity Path (for simple, complete tasks)

This is the "express lane," triggered when the Reasoner determines the entire user goal can be satisfied by a single, self-contained Workflow NLD.

- **Example Utterance:** *"Freeze the user john.doe@example.com."*
- **Reasoner's Action:**
    1. It queries the Knowledge Node and finds a perfect match for the freeze_user.nld (Workflow NLD).
    2. It enters the High-Velocity Path, acting as a simple **translator**. It reads the pre-authored steps from the NLD and directly converts them into a reliable executionPlan.

2. The Dynamic Synthesis Path (for complex tasks)

This is the Reasoner's full "deep thinking" mode, engaged for any request that is more complex than a single workflow. It provides powerful synthesis capabilities for advanced scenarios.

- **Example (Multi-NLD Workflow):** *"Clone user X and freeze user Y."*
    - The Reasoner finds the Workflow NLD for "Clone User" AND the Workflow NLD for "Freeze User." It then **synthesizes** these into a single, two-step executionPlan.
- **Example (Guided Journey / UBER Plan):** *"Help me create a new Flexipage."*
    - The Reasoner identifies a Higher Order NLD that outlines a multi-stage journey. It signals this to the ConversationManager, which then begins an orchestrated, step-by-step workflow with the user.
- **Example (Hybrid Planning):** *"Clone user X and then debug their login issues."*
    - The Reasoner creates a **hybrid plan**. It uses the Workflow NLD for the "Clone" part of the request, and then uses Best Practices NLDs on "Login Issues" to **dynamically generate** a series of diagnostic steps for the second part.

The ultimate objective of the Reasoner is to compile all relevant knowledge, from any and all NLDs, to construct the single most effective executionPlan that completely addresses the user's nuanced request.

1.3 The Dynamic Planning Pipeline: From Goal to *executionPlan*

When the Reasoner Node enters the "Dynamic Synthesis Path," it executes a formal **Plan Generation Pipeline**. This is not a single LLM call, but a sequence of orchestrated actions designed for performance, reliability, and observability. This pipeline is the core of the agent's "deep thinking" capability.

1. **Requirement Synthesis & Clarification**
    - The pipeline begins with the GatherMoreDetails action. This action's sole purpose is to ensure the agent has all necessary information before acting. It analyzes the user's goal against the requirements outlined in the relevant NLDs.
    - If critical data is missing (e.g., "Which user?"), it triggers the **Consultative**

Dialogue Loop, pausing the pipeline and returning a list of questions to the `ConversationManager` to ask the user.

2. **Scaffold Ingestion**
    - Once all requirements are met, the `Reasoner` deterministically ingests the structured `steps` array from the primary `Workflow NLD` or synthesizes a new scaffold from a `Higher Order NLD`. This scaffold serves as the high-level, logical blueprint for the final plan.
3. **Plan Compilation**
    - With the scaffold and requirements in hand, the `Reasoner` invokes its single, powerful **action**. This action encapsulates the entire complex compilation process.
    - The `compilePlan` action's internal logic is a high-performance, two-stage process:
        - **Stage A: Parallel RAG for action Discovery:** It performs **parallel vector searches** for all steps in the scaffold to efficiently find a small, relevant list of candidate actions for each step.
        - **Stage B: Monolithic Compile:** It then uses a **single, well-grounded LLM call** to perform the final compilation. This call is given all the context (requirements, scaffold, and the pre-filtered candidate actions) and is responsible for generating the final, detailed JSON `executionPlan`, intelligently merging steps and ensuring a logical data flow.

Source

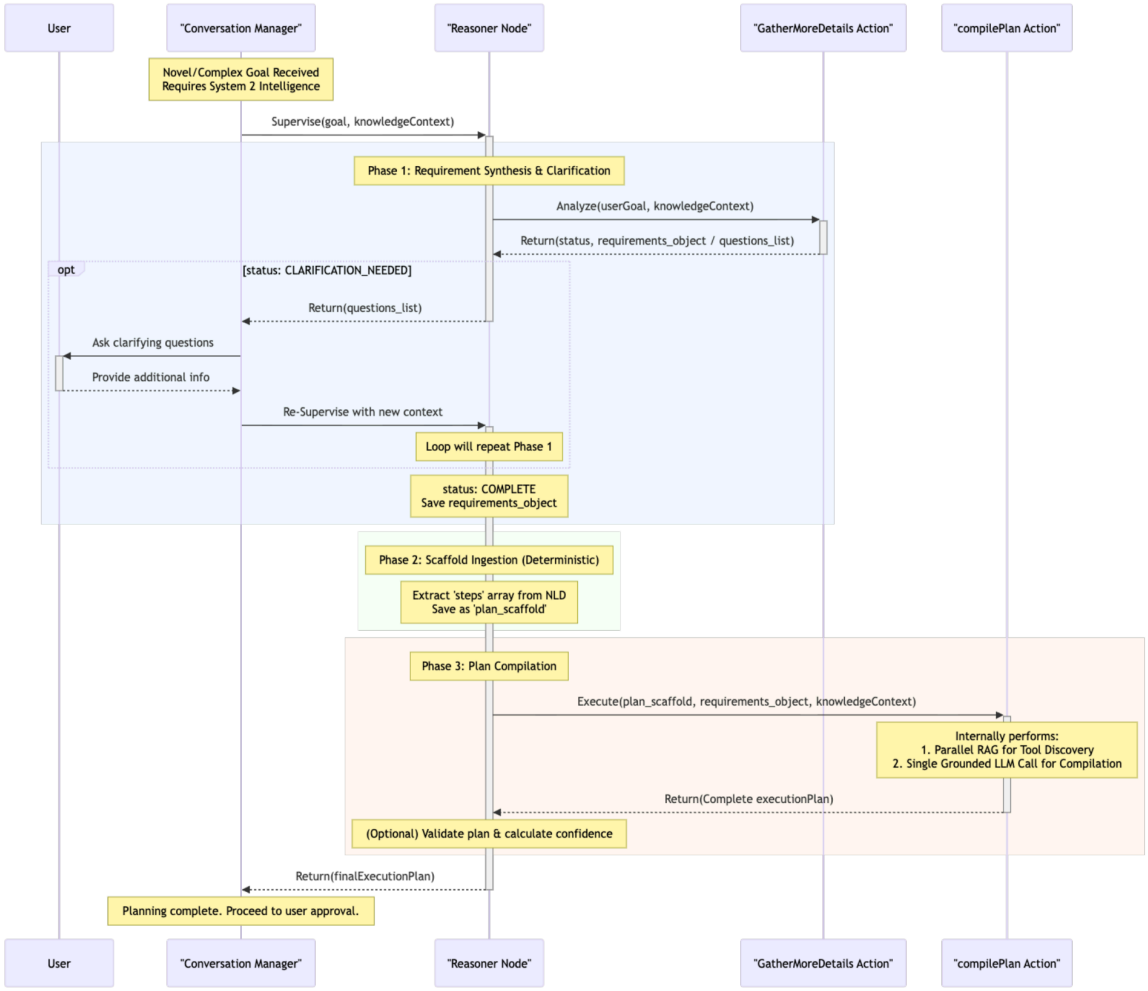


Figure : A detailed sequence diagram of the Reasoner's multi-phase Plan Generation Pipeline, showing the interaction between the Reasoner and its core actions.

1.4 Action Selection: From a Large Catalog to a Precise Choice

A key challenge for any advanced agent is selecting the correct action from a large and growing catalog. A naive approach of showing the LLM all available actions on every turn is slow, expensive, and error-prone. Our V2 architecture solves this with a sophisticated, multi-layered filtering strategy that dramatically simplifies the LLM's decision.

- **How We Do It Today (Bespoke Actions & V1 Limitations):**
    - In the V1 model, action selection is brittle. It relies on the LLM interpreting complex, hand-written instructions in "topics" that attempt to steer it towards a specific, bespoke action (e.g., a hardcoded `ShowActiveUsersAction`). The action catalog is effectively "scoped" by the topic.
- **Where We Are Going:**

- The V2 compilePlan action implements a "funnel" that reduces the decision space for the LLM from hundreds of actions down to a few highly relevant choices.
- **Step 1: NLD-Guided Scaffolding:** The process starts with the NLD steps. This immediately narrows the *conceptual* space. For a "Clone User" NLD, the agent is already thinking about "identifying users" and "creating users," not "updating fields."
- **Step 2: Parallel RAG:** For each step in the scaffold (e.g., "Retrieve the Id of the user"), the compilePlan action performs a **fast vector search** against our entire indexed action catalog. This is the RAG (Retrieval-Augmented Generation) step.
    - This search instantly filters the entire catalog down to a tiny list of the top 3-5 semantically relevant actions (e.g., [IdentifyMetadataRecordByName, QueryRecords, ExecuteEntityRead]).
- **Step 3: The Final LLM Choice (The Fine Filter):** The final compilePlan LLM prompt is then given this small, pre-filtered list. The LLM's job is no longer a difficult "search" problem. It only needs to reason about which of the 3-5 relevant actions is the single best fit for the specific step and the user's requirements.

This funnel-based approach is the key to our scalability. It ensures that even as our action catalog grows to hundreds of actions, the final, critical reasoning step performed by the LLM remains simple, fast, and accurate.

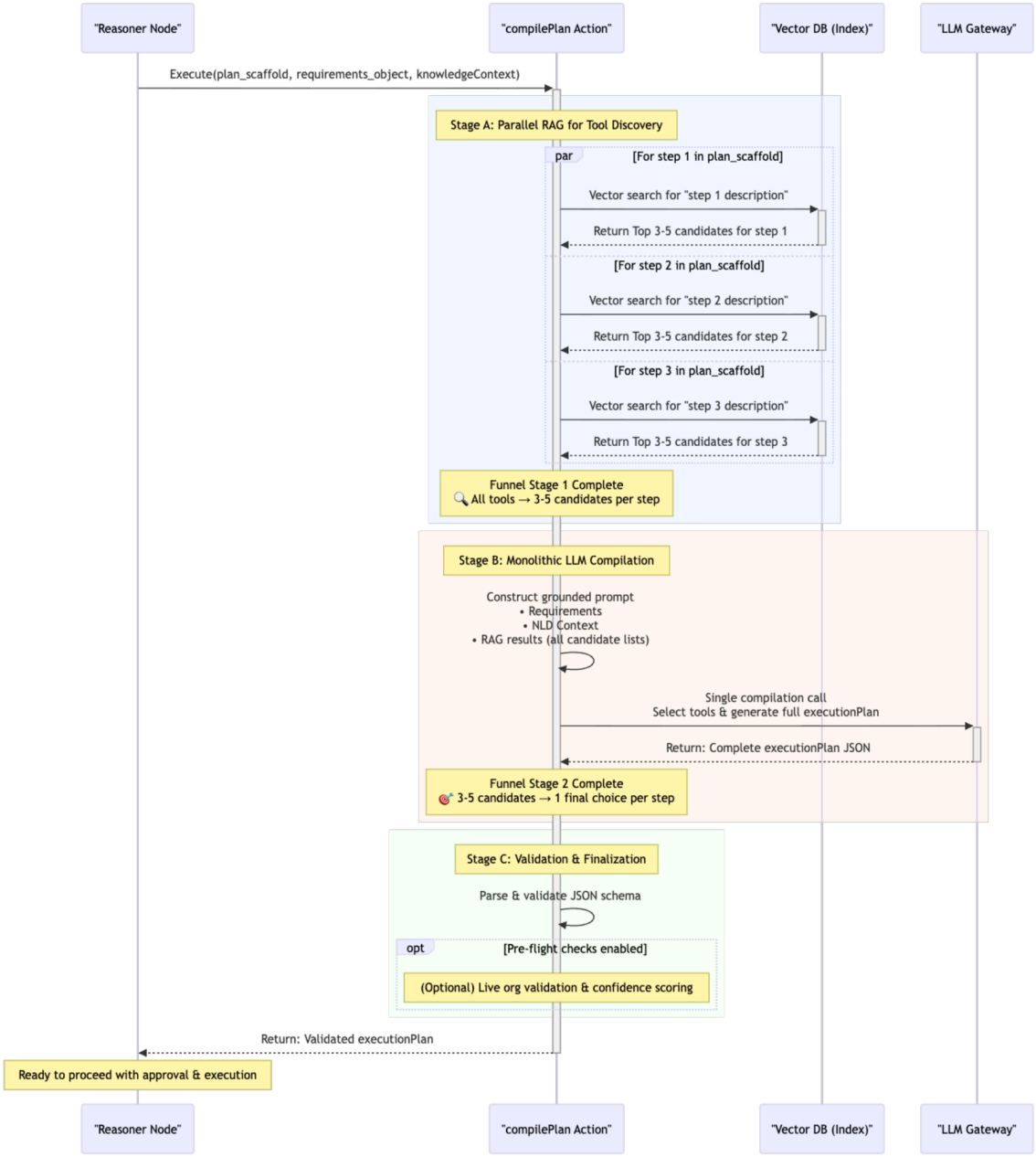


Figure: A detailed sequence diagram illustrating the internal, two-stage 'RAG Funnel' logic within the `compilePlan` action. (Soruce)

1.5 The Predictable Execution Boundary (The Executor's Role)

This node is the cornerstone of the V2 agent's reliability.

- **No LLM:** It contains no reasoning logic. Its control flow is governed entirely by deterministic code.
- **Absolute Fidelity:** It accepts the finalized, fully populated **executionPlan** from the

`ConversationManager` and executes each step with **100% fidelity**, without deviation.
- **State Management:** It manages the *runtime* state of the plan's execution, resolving data dependencies by using the output from a previous step (e.g., a `userId`) as the input for a subsequent step.

Detailed Agent Graph Core Design Agentforce For Setup v2 Core Design

1.6 Strategic Considerations & Future Architecture

Enhancing Agentforce for Setup v2: Strategic Architecture for Performance and Scalability

The "Smart Consultant" vision requires addressing core architectural challenges around **Performance**, **Scalability**, and the **User Experience** (UX), particularly for complex, long-running diagnostic tasks. Our strategy leverages partnerships with the core Agentforce platform to turn these challenges into opportunities.

1. Performance and User Experience: Tackling Timeouts and Latency

The dynamic planning process (specifically, compilePlan) is I/O-bound and can consume significant time, risking a poor UX (a static "Thinking..." screen).

- **Our UX Vision: "Thinking Out Loud"**
  The ideal user experience involves the agent streaming intermediate results (e.g., the high-level plan) in real-time. This dramatically improves perceived performance and builds user trust.
- **Architecture for Streaming**
  Our V2 Reasoner logic is internally designed in a granular, multi-phase manner, making the architecture "streaming-ready." We will advocate for a first-class streaming capability by partnering with the UI Framework and Agent Graph teams.

2. The Serial Execution Bottleneck: A Proposal for Parallelism

The V2 Reasoner is capable of generating complex, multi-step diagnostic plans (e.g., "Show me all permission sets, public groups, and queues assigned to User X"). These plans often contain an initial setup step followed by multiple, **independent data-gathering actions.**

Example of a Serial Plan (Current Model):

1. IdentifyUser (Get User ID).
2. QueryRecords for Permission Sets (uses ID from Step 1).
3. QueryRecords for Public Groups (uses ID from Step 1).
4. QueryRecords for Queues (uses ID from Step 1).

Under the current Agent Graph Executor model, steps 2, 3, and 4 must run **serially**, creating an unnecessary latency bottleneck for common diagnostic scenarios and a suboptimal UX.

Feature Request: Parallel Action Execution

To solve this critical performance issue, we propose a new core platform capability: **Parallel Action Execution**. This feature would allow a plan to declaratively specify that a set of independent steps should be run concurrently, enabling the platform to orchestrate parallel execution.

A. Enhanced Plan Schema
The Reasoner would perform dependency analysis and group independent actions into "parallel execution stages." The executionPlan schema would be updated to support this:

```
None
{
  "executionPlan": {
    "stages": [
      {
        "stageId": "stage_1_find_user",
        "execution_mode": "serial",
        "steps": [ { "stepId": "step_1_find_user", ... } ]
      },
      {
        "stageId": "stage_2_gather_assignments",
        "execution_mode": "parallel", // NEW declarative signal
        "steps": [
          { "stepId": "step_2_find_perm_sets", ... },
          { "stepId": "step_3_find_public_groups", ... },
          { "stepId": "step_4_find_queues", ... }
        ]
      }
    ]
  }
}
```

B. Enhanced Executor Runtime

The core Agent Graph Executor runtime would be enhanced to process this new schema. Upon encountering execution_mode: "parallel", it would submit all steps within that stage for concurrent execution and wait for all to complete before advancing to the next

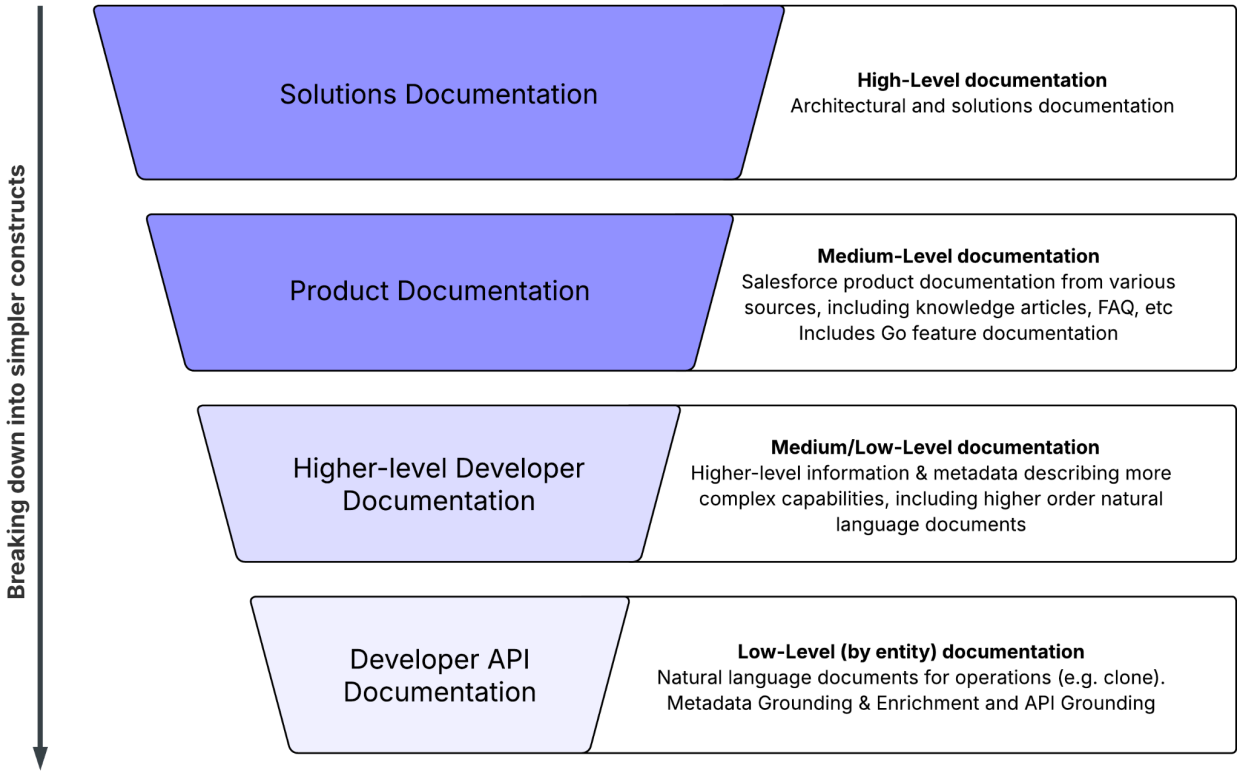
sequential stage.

Pillar 2: The Knowledge Base (Grounding & Ingestion)

The knowledge base is a more basic term we're using to describe a collection of Salesforce information used to ground an Agent on capabilities of the platform. This term is mainly associated with Agentforce for Setup but leverages the main proposal of Salesforce metadata grounding proposed by the Platform Cloud.

This strategy is fully outlined more broadly across the company in the Salesforce Metadata and API strategy document
Salesforce's Metadata and API Strategy for AI Software Construction (AI AppDev)

The knowledge base contains information of various levels ranging from our base API Metadata stack to our more high level product and solution documents. The collection is meant to be a library of information that the agent can use to complete a task within Setup or other Agentic Clients.



2.1 The Knowledge Funnel

Let's break down the different types of information in the knowledge base.

- **Solutions & Product Documents**
  Documents published on help.salesforce.com (and others like

architect.salesforce.com, etc) that are public and accessible to Salesforce Customers. These documents may already outline how to achieve the task within Setup UI. This documentation is useful to an Agent because it may use it to answer information seeking requests; where a user just wants to know how to do it and will manually execute the task on their own. Other documents included here are Knowledge Articles, Help Docs, etc.

- **Natural Language Documents**
  These documents are internal documents that describe the process of achieving the task using our Salesforce APIs. One may describe these documents similar to ones written by an engineer during the design process. Just as an engineer may write a high level design document and then a low-level design document, there are various levels of documents here as well.
    - **Higher Order NLDs** This type of document's main purpose is to provide the agent with information on which direction to take. It should recommend the best features and products for the task, etc.
    - **Workflow NLDs** Once a direction is chosen, the agent can start searching for a step or workflow NLD that will provide a breakdown on how to generate the required pieces listed in the solution.
    - **Best Practices NLDs** These documents can be an extension of API documents. While API documents focus on the schema and APIs for a particular metadata type, best practice documents can be a supplementary document that includes information such as common practices when interacting with the metadata type.
- **API Grounding** A catalog of all the MetadataTypes and their schemas. It also includes API support and example usages. These documents are required per MetadataType if they are exposed in the Metadata API and/or Tooling API.
- **Org Metadata Grounding** This grounding is the other pillar (**Semantic Metadata Grounding)** outlined in the main document. It is customer's org metadata ingested in Data Cloud and available for index text search, vector database search, etc.

2.1.1 NLD Example

Markdown

```
Clone a User

Operation Description

Cloning a user in salesforce creates a new user where most properties are copied from the source and only certain properties such as name, email and username are overriden. The new user can optionally be given the same permission sets and be added to the same groups.
```

Requirements

1. The user to be cloned is provided.
2. Whether to clone with or without assignment is specified. (PermissionSet and GroupMembership)
3. New user details are provided: firstName, lastName, email, username

How to Clone a user via API

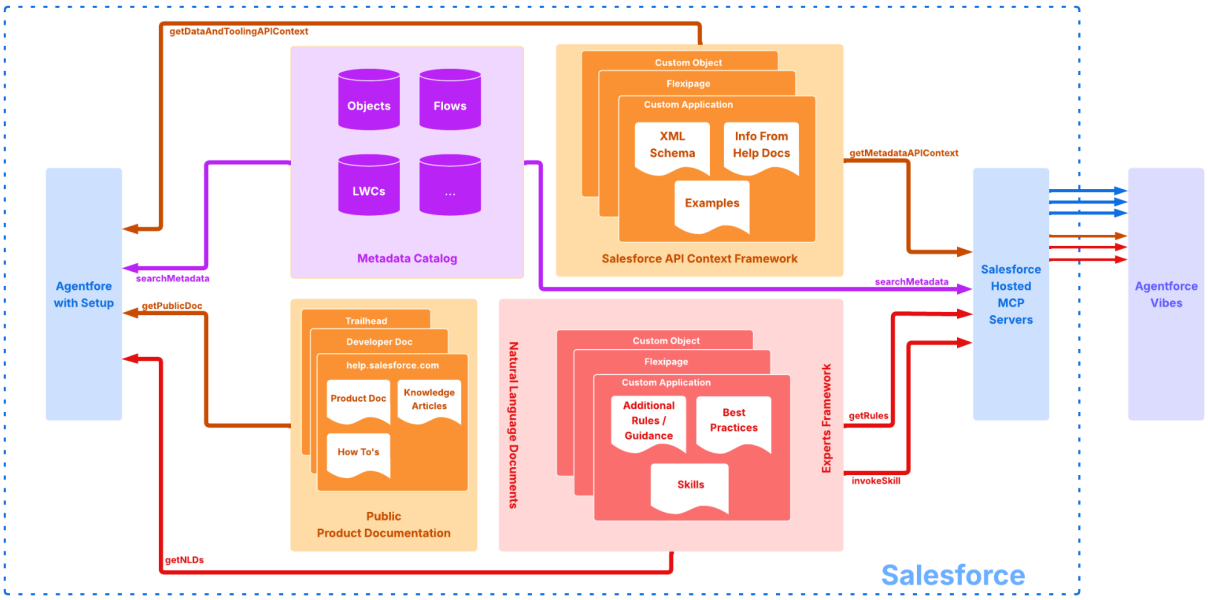
1. Retrieve the Id of the user to be cloned.
2. Create a new User record with the provided new user details.
3. If cloning with assignments is specified
    a. Retrieve the Ids of the PermissionSetAssignments from the user to be cloned.
    b. Create new PermissionSetAssignment records and assign it to the new User
    c. Retrieve the Ids of the GroupMemberships from the user to be cloned.
    d. Create new GroupMembership records and assign it to the new User

JSON Meta Info

```JSON
{
    "name" : "clone_user.md"
    "entities" : ["User", "PermissionSetAssignment", "GroupMembership"],
    "operations": ["Clone", "Clone With Assignment"]
    "permissions": ["Manage Users"],
    "relatedDocuments": ["How to assign a PermissionSetAssignment to a User",
"How to assign a GroupMembership to a User"],
    "scopes" : ["Agentforce for Setup", "Agentforce Vibes"],
    "tools" : []
}
```

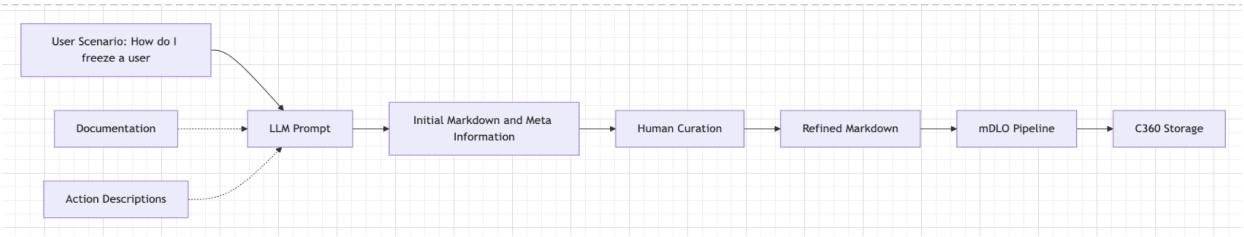
2.2 Alignment with Metadata Experts

Agentforce Vibes uses a similar concept of Metadata Experts which includes similar typed documents that describe a process of interacting with certain metadata types. We believe we can have generic NLDs that allow for sharing of documents between Vibes and Setup. **Discussions are ongoing.**



2.3 NLD Generation Pipeline

In order to simplify the creation of natural language documents, we are leveraging the power of LLMs and ground them via existing data sources. We then validate the content through human curation to make sure the content is accurate. The refined document can then be used to feed the Knowledge DB.



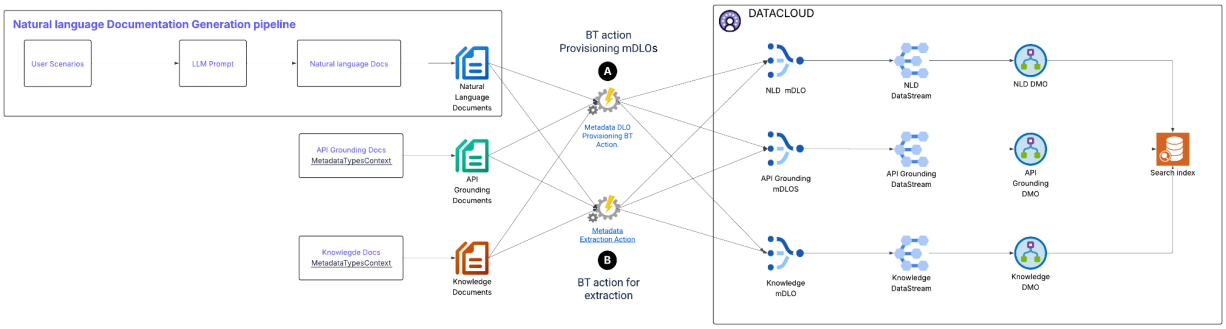
This pipeline automates content creation:

1. Input: User Scenarios (e.g., "How do I create a permission set?").
2. Processing: LLM synthesizes information from Salesforce Help, API Specs and Bespoke Action descriptions.
3. Drafting: Generates a compliant Markdown file with JSON metadata.
4. Human-in-the-Loop: Feature owners review and approve the content.
5. Ingestion: Validated NLDs are then ingested into Data Cloud via mDLOs for retrieval.

More information can be found here: Natural Language Docs Generation

2.4 Knowledge Storage / C360

Knowledge Extraction + Ingestion in C360



The Natural Language, API grounding, and Knowledge documents are all stored in [Markdown (`.md`) format] within Core.

Step A - a one-time setup step is required to provision NLD, Knowledge & API Grounding mDLOs in Data Cloud. **Metadata DLO Provisioning BT Action** process needs to be executed once per every C360 org.

Step B - an end-to-end extraction of data from .md files and ingestion to mDLO's should be triggered via the org-level **Metadata Extraction BT Action** in C360.

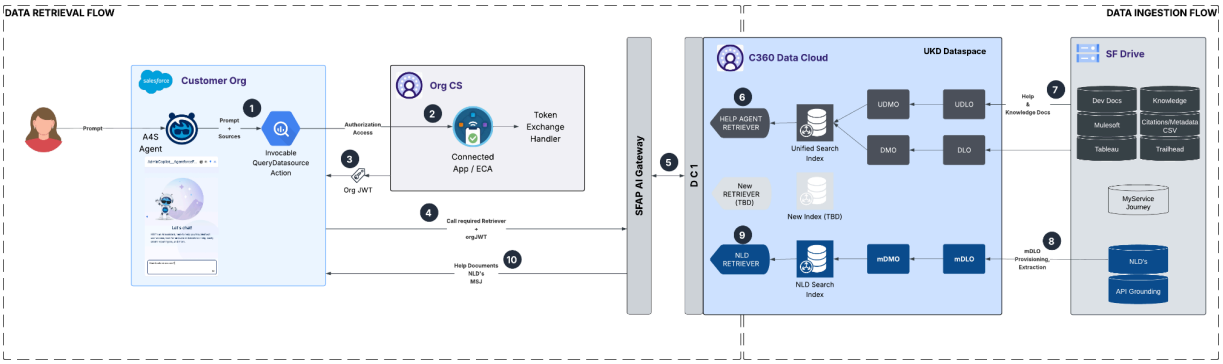
The BT action has two modes: CREATE and READ. In CREATE mode, the action starts an extraction in the background and displays an Extraction Id. The overall extraction's status will change like so:

New → Processing → Syncing → Data_Copied → Ingestion_Requested → Ingestion_Complete

For High level Product + Solution Documentation access : Use existing C360 DLOs from Unified Knowledge dataspace managed by respective content teams. Existing search index with a different retrieval query and appropriate pre-filters will be used.

While NLDs & API Grounding information will be retrieved from a separate index (index created from vectorised DMO's created from mDLO's)

Accessing Knowledge Data from C360



1. **Prompt Initiation:** A user interacts with the **A4S Agent** in the Customer Org, triggering an **Invocable QueryDataSource Action** with a prompt and source from which data is requested.
2. **Authentication Request:** The Customer Org initiates authentication and access via the **Org CS (Connected App / ECA)**.
3. **Token Exchange:** a custom **TokenExchange** handler issues an **orgJWT** token as an **OrgCS/HelpPortal user per Tenant/customer org**, engaging with AF4Setup for further secure connection to C360.
4. **Retrieval Call:** The Customer Org sends a call to the respective **Retriever + orgJWT token** to the **SFAP AI Gateway.**
5. **Gateway Routing:** The **SFAP AI Gateway** routes the request into the **DC1 (Data Cloud)** environment and connects to UKD Dataspace.
6. **Knowledge Retrieval:** Within the C360 Data Cloud (UKD Dataspace), the **Help Agent Retriever** queries the **Unified Search Index** and retrieves relevant chunks.
7. **Data Ingestion (Knowledge):** C360 instance UKD dataspace is the single source of truth for all
8. Salesforce-owned documentations. The extraction is managed by C360 teams.
9. **Data Ingestion (mDLO - NLDs):**
    a. The mDLOs are populated and kept up to date using the Metadata Extraction Framework.
    b. The mDLOs, Transformers, DMOs, Transformers, Indexes etc. are provisioned when an Org Permission is enabled.
    c. The extraction process runs periodically within the core monolith in order to keep the mDLO information up to date.
    d. Services within core query the index in C360 in order to find the DMOs that match a semantic search query.
10. **NLD Retrieval:** The **NLD Retriever** accesses the NLD Search Index (mDMO) to query on NLD's ingested into mDMO's
11. **Data Return:** The retrieved data and relevant documents are sent back through the gateway to the **Customer Org** for further processing by the planner
    a. **Help & Knowledge documents** : Help content is extracted via the same retrieval service used by **Help Agents**. This allows the A4S Agent to access the most

current, verified help content using the same chunking and ranking logic as the core support platform.

b. **NLD's** : The service within core has the option to serve the original source document from the core classpath. This has the potential to cut down on the amount of information that needs to be retrieved from C360 itself.

Pillar 3: Generic Actions (aka tools)

This section provides a breakdown of the Generic API Actions, categorized by their function in the discovery, execution, and display workflow.

Agentforce For Setup v2 Generic Actions Onboarding

1. Discovery Actions

Discovery actions are designed to locate metadata types and records based on user utterances before any modification takes place. In v2, the goal is to consolidate multiple disparate search mechanisms into stable, unified actions. While initially delegating to existing implementations, these actions will eventually swap to using **Grounding APIs** built upon the Unified Catalog Metadata Search (Prism).

To support complex planning where a simple ID lookup is insufficient, a dedicated query action provides raw SOQL execution capabilities.

More information can be found here:
Plans for Consolidating Metadata Discovery Actions

2. Context and Grounding Actions

Grounding Actions provide context to the LLM. The context can be specific to the current organization, i.e. "Does an Apex Class exist for calculating taxes" or organization agnostic, i.e. "What is the API contract for updating an Apex Class".

- **QueryDataSource:** This action replaces the v1 HelpNavigationAction. It is designed to return various levels of knowledge (including navigating information as needed).
  V2 Migration plan for HelpNavigation Action
- **APIGrounding:** This Action provides information that the LLM needs in order to invoke one of the CRUD Execution Actions. The grounding information is sourced from the public API documentation. The Setup Entity must be publicly exposed via the Data or Tooling API.
- **TenantGrounding Action:** This Action provides information about the organization's Metadata by performing a semantic search over a vectorized representation of the organization's Metadata. In the short term this action will still use IdentifyMetadataRecordByName and SetupAgentQueryRecordsNonBillableVersion under the covers

- **ExecuteSOQL Action:** Queries a Setup Entity via the Data or Tooling API with a SOQL snippet provided by the LLM. This snippet typically comes from an NLD Document.

3. Generic CRUD Actions

These actions form the core execution capabilities, allowing the agent to manipulate records via the Salesforce REST API endpoints. They require the Planner to provide the specific API family (Data or Tooling) and the Entity API Name. The planned actions are: **ExecuteEntityCreate, ExecuteEntityRead, ExecuteEntityUpdate, ExecuteEntityDelete**.

More information can be found here: Generic query action proposal

Bespoke UI for Generic Actions

With the shift to Generic Actions, we also get the generic UI for input and output of these actions. This UI may not be the best for every use case. Some teams may want to configure bespoke UIs for input and output of a generic action.

Unfortunately, there is currently no mechanism for this. A rendition can be associated either with a Lightning Type or with an agent action. We will need to figure out *where* to configure such overrides and *how* to specify the component substitution.

An NLD or an equivalent task-specific config seems like the most natural place to store the override. Lightning Types have a concept of channels (e.g. LightningDesktop, LightningMobile, LightningCanvas, Slack, etc) and every channel can have a different rendition of a type. Maybe we could introduce a concept of task-level sub-channels and every type can have sub-channel-specific overrides. Then the task-specific config could indicate its sub-channel.

4. Display Actions and UI Considerations

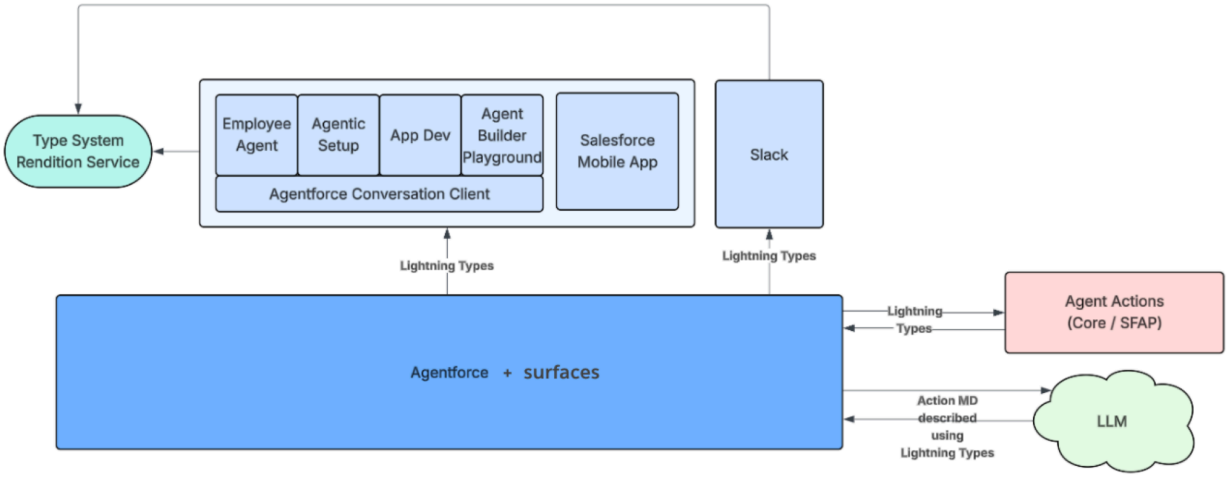
These generic actions are responsible for rendering the results of the agent's work in a feature-rich user interface, rather than just returning raw text.

- **List Support Actions:** Owned by the Lists team, this generic action is designed to return and display multiple records. Lists in Agentforce for Setup V2
- **Record Support Actions:** Owned by the Agentic Patterns & Components team, this provides generic record form support. It is used to render record-like data for a richer user experience.

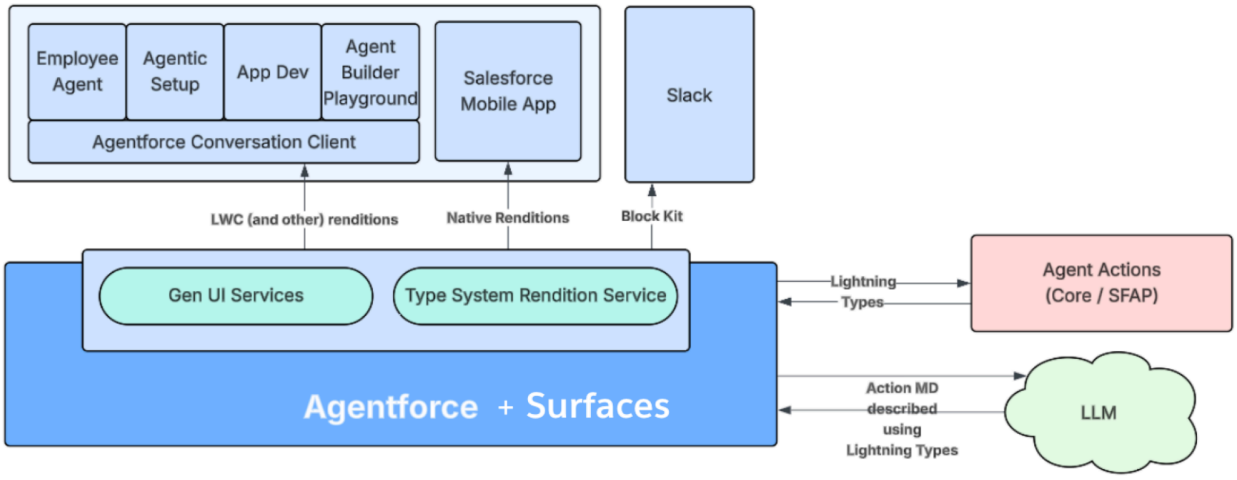
Generating Dynamic UI

There are situations where an agent response might not have a pre-configured UI and we may need to generate a UI layout on the fly. In these use cases, rather than relying on display actions, we are planning to leverage the Agentforce eXperience Layer (AXL) Surface.

Customers interact with Agentforce through different surfaces like LEX, Slack, Mobile etc. A surface is responsible for formatting and outputting the response in ways that match the surface's capabilities and the preferences set by the surface owner or end-user. For example, Slack surface converts Lightning Types to Block Kit and LEX surface just returns Lightning Types as is. Surfaces are built into both Java Planner and Daisy Planner.



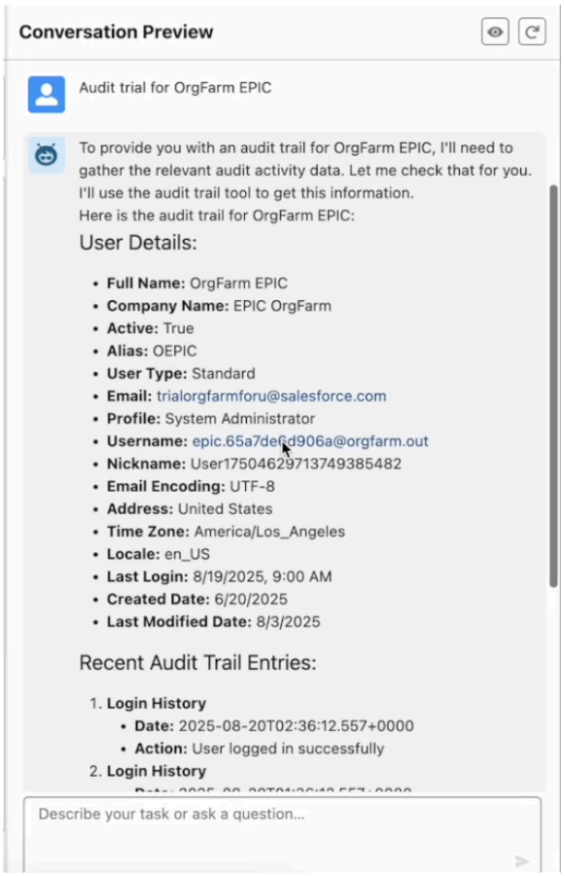
The AXL surface is a new surface which leverages Generative UI Services. These services standardize the process of creating UI metadata with AI. This framework can be used to dynamically compose UI for provided data and user intent. It will compose the UI components from existing components (bespoke and primitive), populate component attributes with data value, and generate the final layout. The selection is made from a surface-specific allowlist of existing components, no new LWC code is generated. The output is a UEM document that can be directly consumed by Agentforce Conversation Client (ACC). LLMs and vector databases are used to efficiently find and configure UI components.



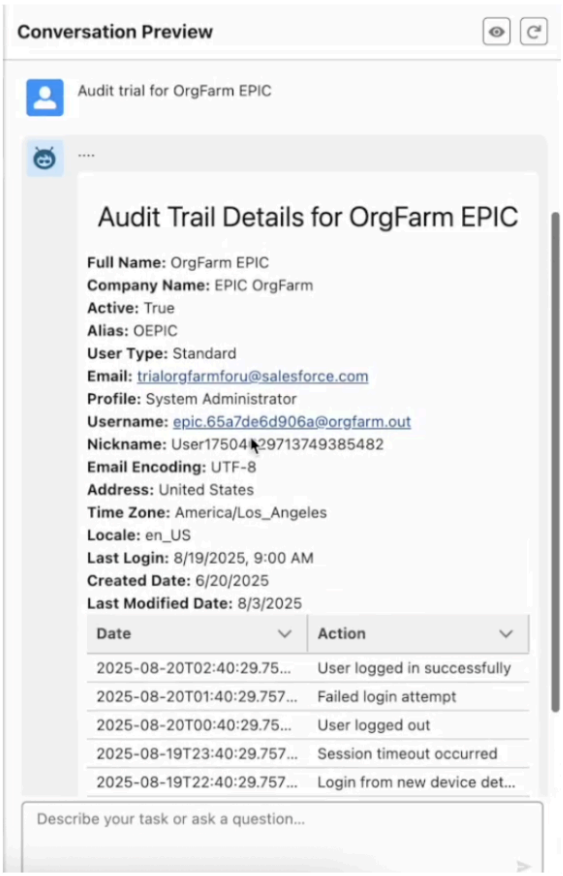
Here's a demo of the AXL surface. For more details on this Generative Services proposal, refer to [Proposal] Generative UI services for Agentforce response rich client rendering

Here's an example of an agent response which was passed through the AXL surface:

Without AXL



With AXL



The AXL surface can help A4S to generate dynamic experiences at runtime:

- Generate a dynamic layout for responses which contain output of multiple actions. For example, a user utterance may require the agent to return outputs of multiple queries. Or we may want to combine multiple Human In The Loop (HITL) confirmation together.
- Detect user preferences in the conversation history and alter the rendition accordingly. For example, a user may ask to see a record table instead of a list of cards. Or see certain columns in a table of records.
- Generate a Dynamic List View from an action output. For example, an action may return a SOQL or a GraphQL query which can be rendered as a List View.

Migration Strategy

The strategy prioritizes a low-disruption transition allowing robust testing.

1. **Co-existence**
   - v2 Agent will co-exist with v1, controlled by a permission gate.
2. **Content Migration**
   - Existing v1 Topics/Instructions will be converted into v2 NLDs.
3. **Action Consolidation**
   - Existing Bespoke Actions will be supported initially to reduce friction.
   - Over time, Bespoke Actions will be retired in favor of the Generic Actions (e.g., UpdateUserRecordFields -> ExecuteEntityUpdate).

More information can be found here:
V1 to V2 Migration Strategy for Setup with Agentforce

Risks

- Knowledge retrieval pipeline from a centralized C360 DC instance is still uncertain, having limited scalability.
- Agent Performance due to the new thinking phase (more accurate) and because the new AgentGraph System currently is 10-20% slower compared to the ReAct Planner. Daisy++ will likely solve this.
- AgentGraph bugs and feature set
   - We keep finding missing features (parity) and bugs within the new AgentGraph stack that delays our progress
   - Daisy++ will likely solve this and solve many of the feature requests we have. Under evaluation.
- DSL File-based Agent support is delayed
   - This heavily impacts developer productivity

Dependencies

- AI Cloud: Daisy, AgentGraph, File-based Agent, Lightning Type Support for Inputs
- Data Cloud: Availability of mDLO ingestion pipelines for NLD
- C360 to provide API and underlying platform support for documentation retrieval

The full list is documented here.

Timeline

The plan is to release the v2 architecture by TDX'26 (April/May 2026)